

Must Know Data Structures & Algorithms

A COMPREHENSIVE INTERVIEW PREPARATION GUIDE

1. Data Structures

Data structures are organized formats for managing, storing, and accessing data efficiently. They are broadly categorized into two types:

Linear Data Structures

Elements are arranged sequentially and linearly, where each element connects to its previous and next element.

Arrays

Linked List

Stacks

Queues

Hash Tables

Non-Linear Data Structures

Elements are arranged in a hierarchical or interconnected network form rather than a sequential sequence.

Trees

Binary Search Trees (BST)

Heaps

Graphs

Trie

Union-Find (Disjoint Set)

2. Algorithms

Algorithms are step-by-step procedures used to perform calculations, process data, and solve computational problems efficiently.

Searching & Sorting

- Binary Search
- Sorting Algorithms

Bit Manipulation

- Bitwise Operations & Patterns

Tree Traversal Algorithms

- In-Order Traversal
- Pre-Order Traversal
- Post-Order Traversal
- Level-Order Traversal (BFS)

Graph Algorithms

- DFS (Depth First Search)
- BFS (Breadth First Search)

- Topological Sort
- Dijkstra Algorithm
- Bellman-Ford Algorithm

3. Problem-Solving Techniques & Patterns

Problem-solving patterns represent common thematic structural frameworks used to solve complex coding interview questions optimally.

1. Two Pointers

Used when working with arrays or strings where two indices move through the data structure to solve problems efficiently.

- Pair sum problems
- Reversing arrays/strings
- Removing duplicates
- Sorted array problems

2. Sliding Window

Used for problems involving contiguous subarrays or substrings to avoid nested loops.

- Maximum/minimum subarray
- Longest substring
- Fixed-size window problems
- Variable-size window problems

3. Prefix Sum

Used to quickly calculate sums of subarrays using precomputed cumulative sums in $O(1)$ time.

- Range sum queries
- Subarray sum problems
- Optimization problems

4. Fast and Slow Pointers

Uses two pointers moving at different speeds (usually one moving twice as fast) to detect structures.

- Detecting cycles in linked lists
- Finding the middle element
- Cycle-related mathematical problems

5. Divide & Conquer

Breaks a complex problem into smaller subproblems, solves them recursively, and then combines their results.

- Merge Sort
- Quick Sort
- Binary Search optimization

6. Greedy

Makes the optimal local choice at every single step with the hope that it will lead to a global optimum.

- Interval scheduling
- Minimum coins problem
- Activity selection

7. Recursion

A foundational problem-solving paradigm where a function solves a problem by calling itself on smaller instances.

- Tree traversal patterns
- Factorial / Fibonacci evaluation
- Base backbone for Backtracking

8. Backtracking

Explores all possible configurations incrementally and backtracks immediately when a path becomes invalid.

- Permutations & Combinations
- N-Queens problem
- Sudoku Solver

9. Dynamic Programming (DP)

Stores the results of overlapping subproblems in a table (memoization/tabulation) to avoid redundant computations.

- Fibonacci optimization
- Knapsack problems
- Longest Common Subsequence (LCS)
- DP on grids and matrices

10. Top 'K' Elements

Used to find the largest, smallest, or most frequent 'K' elements efficiently using Min/Max Heaps.

- K largest elements
- K closest points
- Frequency-based elements sorting

4. How to Master a New Topic

Follow this structured roadmap step-by-step whenever you encounter a brand new DSA concept:

1. Start with Basics

- **What it is:** Understand the baseline concept and purpose of the topic.
- **Code representation:** Learn the syntax, memory allocation, and implementation details.
- **Operations:** Master key structural operations like insertion, deletion, searching, and traversal.

2. Learn Real-World Applications

Understand exactly where and why the topic is used inside real, large-scale software systems.

3. Use Pen & Paper

Dry run problems manually line-by-line to internalize the logic before jumping to an IDE.

4. Implement It from Scratch

Write out the raw underlying implementation details yourself without looking at external code templates.

5. Learn In-Built Libraries

Understand language-provided native libraries, collections frameworks, and built-in utility shortcuts.

6. Solve Small Problems

Build initial confidence by resolving easy-level basic questions before diving into complex problems.

7. Solve Related LeetCode Problems

Practice and reinforce your understanding by filtering specific tagged problems related to the topic.

5. Key Guidelines for Mastering DSA

1. Prioritize solving actual core coding problems over purely sitting and reading theory [cite: 148].
2. Challenge yourself regularly with unfamiliar variants to build problem adaptability [cite: 149].
3. Deeply understand algorithmic concepts and structural trade-offs—never resort to memorizing code [cite: 150].

4. Train your brain to think dynamically in recurring design patterns rather than looking at isolated solutions [cite: 151].